

FPS-3000 SBC Monitor

Operating manual for the ROM-resident monitor, debugger and chassis probe

Revision of 19 August 2026

Contents

1	Getting a prompt	2
1.1	Which image to burn	2
1.2	Serial	2
1.3	Three things that produce silence	3
1.4	The banner	3
1.5	Numbers and arguments	4
1.6	Rehearsing in the emulator	4
2	What the monitor can reach	5
2.1	The chassis in one diagram	5
2.2	The AP I/F	5
2.3	The XLTR	6
2.4	The staging buffer and microcode loading	6
2.5	What no command can reach	6
3	Command reference	7
3.1	Orientation	7
3.2	Memory	8
3.3	Execution	9
3.4	Loading	9
4	Chassis commands	11
4.1	Register maps	11
4.2	The bulk port	12
4.3	The chassis memory window	12
4.4	XP-32 channel transactions	13
4.5	Interrupt taps	13
5	Cautions	15
5.1	Registers that change when read	15
5.2	Registers the monitor deliberately does not write	15
5.3	Errors that must be answered	15
5.4	The timer belongs to the RTOS	15
5.5	Odd addresses	15
5.6	After a fault	16
A	Workspace map	17
B	Quick reference	18

Chapter 1

Getting a prompt

The monitor occupies the tail of the SBC ROM, from F0A826 upward. The firmware ends at F0A825 and everything above it was zero fill, so nothing the firmware needs is moved or overwritten.

It reaches a terminal through the NEC μ PD7201 at F70011 and F70015, channel A. The firmware never touches that chip: a census of every device address the firmware can reach finds no reference to it. That is why the monitor can use the port without coordinating with anything, and also why an otherwise healthy FPS-3000 is silent on it.

1.1 Which image to burn

Two images are prepared, and the choice decides how much of the machine you get.

FPS3K_with_monitor.bin takes the reset vector. The SBC boots straight into the monitor and the firmware never runs. The monitor owns the vectors, the timer and the chassis registers, and every command works. Use it for serial bring-up and for any session that drives the chassis by hand.

FPS3K_panic_only.bin leaves the reset vector alone and replaces the firmware's exception catch-all at F0A27A. The machine boots normally, runs its self-test, starts RMS68K and its six tasks. If it would have panicked you get a prompt instead, with the faulting registers intact. Use it to inspect a running machine and to see the chassis in the state the firmware left it.

Three things differ between the two images, because they involve something the firmware is using. `s+` refuses to program the timer when the RTOS owns the tick. `t` borrows the trace vector, which the kernel fills, and puts it back. And FF021A reads \$0000 on the reset image against \$0FFF on the panic image, because the self-test is what writes it. Each is noted where it arises.

1.2 Serial

9600 baud, 8 data bits, no parity, 1 stop bit, no flow control. The rate is strapped on the VM02 and cannot be changed in software on this board revision.

Send a bare carriage return, not CR LF. The line reader accepts either terminator, so CR LF ends the line on the CR and then starts and ends an empty one, which draws a second prompt. From that point every command reads back the previous command's reply, and

nothing announces that it has happened. The symptom is replies that lag one command behind.

Serial reaches the board through P2 on the backplane, not a front-panel port. Three wires are enough.

P2 pin	Direction	Signal	Adapter side
73	SBC out	TXD1	RX
75	SBC in	RXD1	TX
1 to 6		GND	GND

The signal names are from the SBC's point of view, so this is an ordinary null-modem arrangement. Some USB adapters want DSR or DCD asserted before they will transmit. Tie the adapter's DSR and DCD to its own DTR, or wire SBC DTR1 on pin 87 to the adapter's DSR.

1.3 Three things that produce silence

Each of these has happened on this machine, and each is indistinguishable from a dead board.

The RS-232 drivers may be unpowered. The 1488 and 1489 line drivers need plus and minus 12 volts, and the FPS-3000 chassis does not supply them. The backplane pins are present and nothing drives them. Feed them from an external supply.

Jumpers J25 and J26 must be fitted. They connect the serial signals to P2. NASA's RTMPS machine removed them deliberately, and their own documentation notes that doing so "effectively removes all serial I/O signals from the P2 connector of the board". Both are fitted on this chassis's card.

TX and RX may be swapped. See the table above.

1.4 The banner

```
=====
FPS-3000 SBC Monitor / Debugger
Lives in 21.9 KB free ROM @F0A826
Talks via SIO chA (F70011/F70015)
=====
entered at PC=$00000000 SR=$2700
fps3k>
```

The banner's "21.9 KB" is the size of the free region the monitor was written into. The monitor is 10,842 bytes of it, ending at F0D27F. F0D280 to F0FFFD is still zero, 11,646 bytes, and the last word of the ROM holds the checksum.

On a fault entry the last line carries more:

```
entered at PC=$00001006 SR=$2700 FAULT@$00001001 SSW=$0015
```

FAULT@ is the address that could not be completed. SSW is the special status word: bit 4 set means a read, and bits 2 to 0 are the function code, where 5 is supervisor data and 6 is supervisor program. Both appear only for a bus or address error, since an ordinary exception frame does not carry them. The i command prints the same two fields unconditionally, so after a clean entry it shows them as zero.

1.5 Numbers and arguments

All numbers are hexadecimal, with no prefix. Case does not matter. Arguments are separated by spaces or tabs.

Command lines are limited to 61 characters. A longer line is refused outright and nothing is executed. This matters because the natural way to plant a test stub is a long `ww` line, and a silently truncated one would land mid-token and write a wrong value.

1.6 Rehearsing in the emulator

The project's emulator implements the same serial controller and routes channel A to standard input and output.

```
cd emulator
./fps3k_sbc -rom ../monitor/FPS3K_with_monitor.bin
```

Do not pipe a command in as one burst. The emulated receiver holds a single character, so a whole line written at once arrives as its last character only and the monitor sees an empty line. Type interactively, or drive it from a script that writes one character at a time with a short pause between them. `tools/fps3k_probe.py --emulator` does exactly that and is the easier route for anything repetitive.

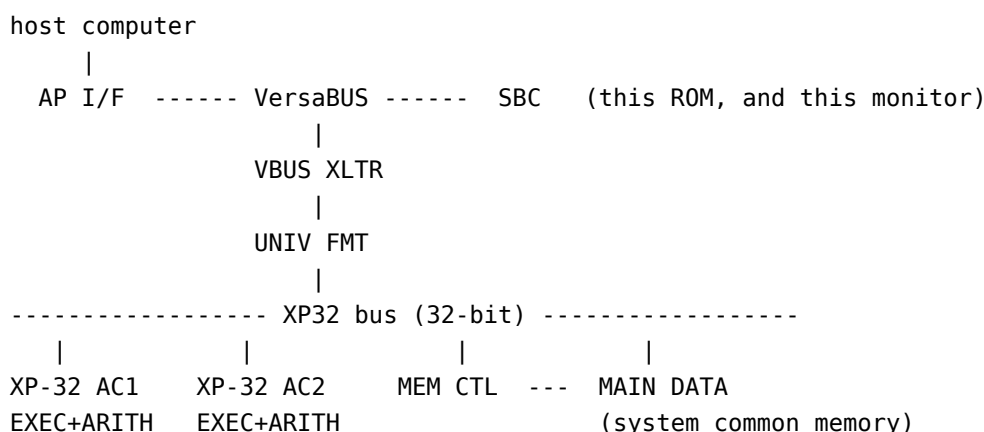
The emulator answers the chassis registers from a model rather than from hardware. It is the right place to check that a sequence of commands does what you meant, and the wrong place to conclude anything about what is fitted. The `c` command says in its own output when it is looking at a latch rather than at a coprocessor.

Chapter 2

What the monitor can reach

This is a short orientation to the parts of the machine the monitor can address, and to the one part it cannot. It is not a description of the FPS-3000. Read it before the command reference and the register names there will mean something.

2.1 The chassis in one diagram



The SBC is the control processor. It does integer work, addressing and sequencing. The floating-point work happens in the XP-32 arithmetic coprocessors, each of which is a pair of cards: an EXEC card carrying the control microcode and an ARITH card carrying the floating-point pipes and their writable control store.

This chassis is populated with two coprocessors. The firmware exposes four channels generically, and the two that are not fitted gate themselves off by reading a channel-presence count the firmware builds at boot by probing all four command ports. The x command reads those same four ports, so it answers directly whether a given chassis has one coprocessor or two.

2.2 The AP I/F

The AP I/F is the host interface. It presents the SBC with a bulk window and four channel windows.

The bulk window is a word count at FF0000, a ready flag at FF0004, a data FIFO at FF0008 whose reads pop, and a command port at FF000E.

Each channel window is a write-once register at offset 4, a 32-bit data register split across offsets 8 and 10, and a command and status register at offset 14. The two data halves are one register, not a data port and a status port. The firmware writes both and reads them back as a single longword.

The card's host side is not a parallel 32-bit bus. It carries two quad differential drivers and two quad receivers, so at most eight channels each way, some of which are clock and handshake. The 32-bit width lives in the eight dual-port memories on the card, and the cable is time-multiplexed. A counterpart card built from the phrase "32-bit interface" would be badly wrong.

2.3 The XLTR

The XLTR is the SBC's control panel for the chassis. The `y` command dumps it.

The registers you will meet most are the two mode registers at FF0200 and FF0202, the channel select register at FF0204, the page register MODE2 at FF0210, the status and interrupt-arm register at FF0218, and the interrupt mask at FF021A.

The channel select register is the busiest register on the board, with about 33,000 writes in a single boot. Outbound it carries only status: the self-test broadcasts its phase number there, which is why a stalled board leaves a readable phase code in it. Inbound it carries every parameter the chassis protocol has, one 16-bit half at a time.

The three MC68153 interrupt modules supply the vectors for the channel and host interrupts. Six of their twelve channels are live.

2.4 The staging buffer and microcode loading

SBC RAM from 010000 to 01FFFF is the staging buffer for coprocessor microcode. It is 64 KB, which is exactly one bank of the ARITH card's writable control store: 32 static memories, four bits wide, 4K deep, giving 4K words of 128 bits.

A microcode load is not the SBC copying the buffer to the coprocessor. The SBC sends an address and a count, and the coprocessor fetches. The chassis is a bus master. The `L` command writes into that buffer directly, which is the same destination the firmware's own `S`-record path reaches, without the chassis-side dispatch.

The top of the buffer is not free. Live kernel structures occupy 01DD00 upward, including the six task control blocks. The firmware's own bounds check permits the whole range, so a record addressed past 01E8F0 passes the check and corrupts kernel state. The monitor's loader refuses only its own region and the vector table, so this hazard is yours to avoid.

2.5 What no command can reach

The link between an EXEC card and its ARITH card is four 40-pin ribbon cables running directly between the two boards, 160 conductors. It is on neither the VersaBUS nor the XP32 bus. The self-test never touches EXEC, ARITH or the format converter, and the firmware's entire vocabulary toward a coprocessor is five operation codes and three verbs.

This is a property of the machine, not a gap in the monitor. No amount of probing from the SBC can observe what happens on that ribbon.

Chapter 3

Command reference

3.1 Orientation

h, ?

Print the command list.

!

Reprint the banner.

i

Print the memory map and live diagnostics.

```
RAM:      128 KB (000000-01FFFF)
ROM:      64 KB (F00000-F0FFFF)
WCS buf:  010000-01FFFF (fully loadable via L)
mon work: 00F800-00F8FF vars/bp, 00F900-00FB7F tap ring
          stack top 00FF00 (896 B below the ring)
AP I/F:    FF0000-FF00FF
XLTR:      FF0200-FF025F
SIO chA:   F70011 data / F70015 ctrl (odd bytes)
PTM:       F70001-F7000F (odd bytes)
board status F70019 = $1F
VMOD ctrl  1FFF0  = $0000
monitor_end      = $00F0D280
grp0/nest/txfail = $FF/00/00  FAULT@$00000000  SSW=$0000
```

The last four lines are the interesting ones on a bad day. The board status register and the VMOD control image are read live. The three diagnostic bytes report the kind of exception frame the monitor was entered with, whether it is currently reporting a fault, and whether a serial transmit has ever timed out.

Unlike the banner, **i** prints the fault address and status word unconditionally, so on a clean entry they read as zero.

r

Display the saved registers: D0 to D7, A0 to A6, then PC and SR. On a fault entry these are the faulting task's registers. From cold entry they are whatever the monitor initialised.

3.2 Memory**m AAAA [NN], d AAAA [NN]**

Dump NN bytes, default 16, with an ASCII column.

```
fps3k> m F00000 10
00F00000: 00 00 00 00 00 00 F0 A8 26 00 00 00 00 00 00 00 | .....&..... |
fps3k> m
00F00010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | ..... |
```

A bare m continues from where the last one stopped, as above.

That first line is worth reading. The 68000 takes its initial stack pointer from the first long-word of ROM and its initial program counter from the second, and here the second reads \$00F0A826. This is the reset image, so the monitor has the machine from power-on.

mw A [NN], ml A [NN]

Dump NN words or longs. Odd addresses are refused with a message rather than faulting, since a word access to an odd address is an address error on a 68000.

These exist because the AP I/F and the XLTR are 16-bit blocks. A byte access to them does not work, so m cannot read them and mw can.

w AAAA BB BB ...

Write bytes.

ww A VV ..., wl A VV ...

Write words or longs. Odd addresses refused as above.

f AAAA NNNN VVVV

Fill NNNN words with VVVV. Bounds-checked by the same routine the S-record loader uses, so it will not write over the vector table, the monitor's own workspace, or anything outside RAM. For deliberate device pokes use w, ww or wl.

z AAAA NNNN

Compute both the exclusive-OR and the additive sum of NNNN words, in one pass.

```
fps3k> z F00000 10000
XOR=$0000 SUM=$17B71936
```

This is the firmware's own ROM criterion. Phase \$0300 requires the exclusive-OR of the whole ROM to be zero, so z F00000 10000 needs no expected value supplied from outside: the machine itself declares what the answer must be. The sum is not a fixed number and will differ for any other image.

The additive sum runs alongside because exclusive-OR alone cannot detect reordering. Two regions with the same bytes in a different order have the same XOR and different sums.

At 9600 baud this is the difference between one round trip and about six minutes. Dumping 64 KB to check it by eye is roughly 512 round trips.

3.3 Execution

g [AAAA]

With no argument, resume from the saved exception frame. This is how you continue after a breakpoint or a recoverable fault.

With an address, start execution there. The monitor synthesises a frame. This works from cold entry and after a bus or address error, where no resumable frame exists. A group-0 frame is discarded when you do this, so repeated fault-then-restart cycles do not leak stack.

Bare g refuses if the frame is not resumable, and says so.

b [ADDR | -ADDR | -]

Breakpoints, up to eight. b ADDR sets one, b lists them, b -ADDR clears one, b - clears all.

A breakpoint replaces the word at the target with a TRAP #14 instruction and keeps the original in its slot. The trap vector is not claimed by the kernel, so the monitor can install it permanently and leave it there.

Targets are bounds-checked. The address must be even, non-zero, in on-board RAM, and outside the monitor's own region. Setting one inside the monitor's stack would work briefly and then be overwritten by the next deep call, and clearing it afterwards would write the saved word back over live stack.

Execution stops one word past the trap, which is where the 68000 leaves the PC.

t

Single step one instruction using the trace bit in the status register.

The trace vector is borrowed, not taken. Unlike TRAP #14 the kernel does own vector 9, its static vector table puts \$F00AEE there, so the monitor saves the original on first use and restores it on the next g. Both the vector and the trace bit are put back, on both the bare and the addressed form of g.

3.4 Loading

L

Receive Motorola S-records. Types S0 through S3 are accepted, S7, S8 or S9 ends the load. Press ESC to abort.

```
fps3k> L
```

```
send S-records, S8/S9 ends, ESC aborts:
```

```
.
```

```
loaded $00000001 records, $00000004
```

```
fps3k> m 2000 8
00002000: DE AD BE EF 00 00 00 00 | .....|
```

The records themselves are not echoed. One dot is printed per record whose checksum verifies, so a long transfer shows progress. The two records sent above were S1072000DEADBEEFA0 and S9030000FC.

Records are read straight from the serial port rather than through the line buffer, so a file can be streamed at line rate.

Each record's checksum is verified. Data is written before the checksum can be computed, so a record that fails may have been partly applied. Resend it.

ESC aborts. This matters because the character reader blocks, and before the abort existed a mistyped L or a sender that died mid-transfer left the monitor waiting for a character that would never arrive, recoverable only by a power cycle. ESC is safe to use for this because a valid S-record stream contains only S, hex digits and line terminators.

Refused ranges:

- 000000 to 0003FF, the exception vector table.
- 00F800 to 00FEFF, the monitor's workspace and stack.
- anything at or above 020000, which is ROM and peripherals. A stray record aimed at F70011 would transmit a byte out of the serial port, and one aimed at the XLTR would poke the chassis.
- any record whose tail runs past 01FFFF, even if its start address is legal.

A refused record is reported and skipped, and the load continues.

Chapter 4

Chassis commands

The commands in this chapter drive real hardware. Chapter 2 is the map they work against.

4.1 Register maps

x

Dump the AP I/F registers.

AP I/F -- bulk window + 4 channel windows

(window N base = FF0000 + ((N+1)<<5), so FF0020 is skipped by construction)

The card presents a uniform grid of 32-byte windows of which five are used: one bulk window at FF0000 and four channel windows at FF0040, FF0060, FF0080 and FF00A0. The firmware's own arithmetic is $(ch + 1) \times 32$, which is why FF0020 is skipped: it is skipped by construction, not by omission.

The command walks a table of addresses established by a census of the firmware rather than sweeping the block. Sweeping would be worse than useless, because an address nothing answers raises a bus error and aborts the dump.

Two addresses are deliberately absent:

- FF0008 is the bulk data FIFO. Reading it pops it. Use e.
- FF0010 is never accessed by the firmware in any way. It is a register the emulator invented, not one awaiting confirmation.

y

Dump the XLTR control block and the three MC68153 bus interrupt modules, 34 registers in all.

The XLTR is the card that carries the SBC's view of the chassis: mode registers, the channel select register, the status and interrupt-mask registers, and the page register for the chassis memory window.

4.2 The bulk port

e [NN]

Drain NN words from the bulk data port at FF0008, reading the word count at FF0000 before and after.

```
arming FF0218 <- $400, polling bit 15...
FF0000 before drain = $0100
FF0000 after drain = $00FC
-> COUNT FELL: FF0000 behaves as a hardware down-counter
```

The firmware compares FF0000 and never writes it, and the card carries six up/down counters against three up-only ones. The expectation is that it is a hardware down-counter loaded from off-card, decrementing as the FIFO is popped. This command tests that directly.

In the emulator the count does not fall, which is the correct answer there: the model has no counter behind that address. This is a test the emulator can only fail.

q

Poll bit 0 of FF0004 a bounded number of times and report.

```
polling FF0004 bit 0 (BOUNDED -- the firmware's own 11 sites are not)
bit 0 SET after $00000042 polls, FF0004=$0001
```

The firmware polls this flag at eleven sites and every one of them is an unbounded spin with no timeout and no exit but success. If the flag never sets, the firmware stops, with no diagnostic at all. The monitor's version terminates in under a second so you can find out what the flag is doing without hanging the board.

4.3 The chassis memory window

p PP [NN [OO]]

Read NN longs, default 32, from page PP of the chassis window at offset OO.

pw PP OO VV ...

Write consecutive longs at page PP, offset OO.

The window at 400000 is paged by the XLTR's MODE2 register. The effective address is the page in the top bits and the offset below, giving a 4 MB byte window per page and 4096 pages.

Both commands save MODE2 on entry and restore it on exit. This is not housekeeping. MODE2's resting value after boot is zero, set twice by the firmware and never changed afterwards, and all three firmware paths that touch it save and restore. A monitor that left another page selected would leave the RTOS looking at the wrong page with nothing to correct it.

The two-step alternative, writing MODE2 by hand and then reading the window, reproduces exactly the fault those three disciplines exist to prevent.

The offset argument on p is third rather than second on purpose. Putting it second would silently reinterpret an existing p 0 4 as page 0 offset 4 rather than page 0, four longs.

Offsets at or above 100000 raise a bus error in the emulator, whose window is modelled as 1 MB. On hardware a page is 4 MB. The monitor addresses the architectural window, not the model's.

4.4 XP-32 channel transactions

c CH OP [DATA]

Run a channel transaction by hand on channel CH, 1 to 4, with operation code OP

Without DATA this writes zero to the data-high register, the operation code to data-low, issues REQUEST-TRANSFER (\$8004) to the command register, and polls for the DONE bit. The poll budget is 1000 iterations, the same as the firmware's.

With DATA it continues into the second phase, writing the longword across the data pair and issuing CONTINUE-TRANSFER (\$8005). A normal channel request is two back-to-back transactions, so a command that can only issue the first cannot reproduce one. The continue phase is gated on the first reaching DONE.

```
fps3k> c 1 10 DEADBEEF
DONE  status=$C004  polls=$00000000  data=$0000:0010
  ** LATCH ECHOING, not an AC responding: zero polls and
    +$0A read back the opcode we wrote.  Vary OP and compare.
CONTINUE ($8005): DONE  status=$C005  polls=$00000000  data=$DEAD:BEEF
BIM CR restored; FF021A left alone, was $0000
```

The latch warning is the important part of this command. A status that is identical for every operation code, completing in zero polls, and handing back the word just written, is a latch echoing rather than an arithmetic coprocessor answering. Without that discriminator a passing emulator run reads exactly like a live AC.

That capture is from the reset image, where the self-test has not run, so the mask register reads zero. On the panic image the self-test leaves it at \$0FFF and the last line will say so.

The command masks the channel's interrupt module for the duration and restores it on every exit. It reports the interrupt mask register at FF021A and deliberately does not write it. See chapter 2.

ca CH OP

Issue the acknowledge verb \$8000 and do not poll.

This is the SBC answering the coprocessor rather than asking it. The firmware's three sites for it branch away immediately with no poll and no status read, because the chassis owes no response. Polling here would time out on correct hardware and report a fault that is not one.

4.5 Interrupt taps

s, s-, sl, sc, s+

Capture channel interrupts into a 64-entry ring.

- s arms taps on vectors 45 to 48, chaining to the original handlers.
- s- disarms and restores the original vectors.
- sl lists the ring.

- sc clears it.
- s+ programs the timer for timestamps, on the reset image only.

```
CH TSTMP STAT DHI :DLO
01 4E20 C000 0000:001B
02 4E9C C000 0000:001B
wraps=$0000
```

Each entry records the channel, a timestamp from timer 3, and the three words the interrupt service routine latched: status, data high, data low.

Arming twice is refused. Doing it would save the taps over the original vectors, and the disarm would then restore the taps over themselves. On the panic image the result is not untidiness but a hang, since the chained handler would be the tap itself and the interrupt would never be acknowledged.

s+ is refused when the RTOS owns the tick. The first thing it writes is a chip-wide internal reset that holds all three timers, and timer 3 is the 10 ms system tick. On the panic image the RTOS has already programmed it, so timestamps work without s+.

Chapter 5

Cautions

5.1 Registers that change when read

FF0008 is a FIFO. Reading it pops a word. The x command skips it for that reason, and e is the command that reads it deliberately.

5.2 Registers the monitor deliberately does not write

FF021A, the interrupt mask, is clear-only in the firmware. Fifty sites read it, clear one bit, and write it back, always six instructions apart, and nothing ever sets a bit. The c command displays it and does not write it back.

Writing the whole word back would be the only operation in the machine capable of raising a bit rather than lowering one. While your own channel's interrupt module is masked, the other three are still live at level 7 and their handlers may legitimately clear their own bits between your read and your write. A blind write-back resurrects them, and the consequence is a channel that never re-enables, which looks like a firmware bug.

5.3 Errors that must be answered

If a channel raises an error and nothing answers the resulting report, that channel's bit stays set in FF021A and its interrupt module stays masked. It never re-enables. A machine degrades one channel at a time this way.

5.4 The timer belongs to the RTOS

Timer 3 of the MC6840 is the 10 ms system tick. Programming it stops the kernel's sense of time. s+ refuses to run when the tick vector does not point at the monitor, which is the test for whether the firmware or the monitor owns the machine.

Note also that the control register bit used to set up a timer is a chip-wide internal reset. It holds all three timers, not just the one being programmed.

5.5 Odd addresses

Word and long accesses to odd addresses are address errors on a 68000. The monitor refuses them with a message. The serial controller and the timer are both odd-byte devices, so byte

commands are the right tool there and the width commands are the right tool for the chassis blocks.

5.6 After a fault

A bus or address error leaves a frame that cannot be resumed. Bare g refuses it. g ADDR starts fresh and discards the frame properly.

If MON_NEST reads 2 in the i output, the monitor faulted while reporting a fault. The first fault's PC and status are still intact, which is the point of the guard.

Appendix A

Workspace map

The monitor's variables live at 00F800 through 00FEFF, with the cold-entry stack top at 00FF00. Measurement of a full stock boot shows 001200 to 00FFFF untouched by the firmware, which is why the workspace is there.

Address	Size	Contents
00F800	60	saved D0 to D7, then A0 to A6
00F83C	4	saved PC
00F840	2	saved SR
00F844	4	command-loop stack anchor, used by the ESC abort
00F848	1	ESC abort flag, set only during L
00F84C	4	workspace magic, MON1 once initialised
00F850	64	command line buffer
00F890	4	last dump address
00F894	1	frame kind: 0 short, 1 group-0, FF none
00F895	1	re-entry guard
00F896	2	transmit-timeout flag
00F898	4	fault address, group-0 only
00F89C	2	special status word, group-0 only
00F8A0	48	breakpoint table, 8 slots of address and saved word
00F8D0	1	armed tap mask
00F8D4	4	saved trace vector, 0 if none
00F8D8	16	saved channel vectors
00F8E8	6	ring head, count, wrap count
00F8F0	16	tap chain targets, one per channel
00F900	640	capture ring, 64 entries of 10 bytes
00FF00		stack top, growing down

The workspace is protected from the S-record loader, the fill command and the breakpoint setter by a single shared bounds routine. Three commands calling one routine is deliberate: every time this monitor grew a second copy of a rule, the copies diverged.

Appendix B

Quick reference

r	registers		
m/d AAAA [NN]	dump bytes	w AAAA BB ...	write bytes
mw A [NN]	dump words	ww A VV ...	write words
ml A [NN]	dump longs	wl A VV ...	write longs
f AAAA NNNN VV	fill words	z AAAA NNNN	XOR + sum
g [AAAA]	resume or start	t	single step
b [A -A -]	breakpoints	L	load S-records (ESC aborts)
x	AP I/F map	y	XLTR + BIM map
e [NN]	drain bulk port	q	poll FF0004 bit 0
p PP [NN [00]]	read chassis page	pw PP 00 VV ...	write chassis page
c CH OP [DD]	AC transaction	ca CH OP	AC acknowledge
s [- l c +]	interrupt taps		
i	status and map	!	banner
h, ?	help		

All numbers hexadecimal. Lines limited to 61 characters. Send bare CR.